The Title is Not Dog Treats

By

© 2026

Athena Harris

Bachelors of Science in Computer Science, Kansas University, 2024
Bachelors of Science in Mathematics, Kansas University, 2024

Submitted to the graduate degree program in Your Department and the Graduate Faculty of the University of Kansas in partial fulfillment of the requirements for the degree of Your Degree.

_____

Chair: Perry Alexander

_____

Member 2

_____

Member 3

_____

Member 4

_____

Member 5

Date defended: _____MM DD, YYYY_____

The Thesis/Dissertation Committee for Athena Harris certifies
that this is the approved version of the following thesis/dissertation :

The Title is Not Dog Treats

_____

Chair: Perry Alexander

Date approved: MM DD, YYYY

Abstract

Formal verification is a useful tool in guaranteeing the security and behavior of programs. This is increasingly true as the amount of LLM generated code continues to rise. However even simple projects can be labor intensive to verify, thus verification see's little use in practice. Existing methods have either used program synthesis algorithms or LLM's to automate verification. Now applied blindly and on entire proofs, LLM's lack much of the resources that programmers are able to utilize and thus are not as effective in writing proofs. We present a method for introducing relevant information from the environment to the LLM, while using it to develop each proof iteratively and a tool based off this method called NotDogTreats. This consists of tracking and ranking information in the local context and providing only the most relevant information to the LLM. The process then iteratively attempts to solve each part of a proof one step at a time, allowing us to see if we are making progress. [Discuss Evaluation once Evaluated]

Acknowledgements

This is where you will insert you acknowledgements. Usually in this section, students will mention their thesis/dissertation chair and committee, any research associates, their friends, and their family members. But who you thank is ultimately up to you.

Contents

List of Figures **7**

List of Tables **8**

1 Introduction **9**

1.1 Overview 9

1.2 Background 10

2 Literature Review **12**

2.1 Name of Subsection 12

2.1.1 Name of Third-Level Subsection 13

2.1.1.1 Name of Fourth-Level Subsection 13

3 Methodology **14**

3.1 DogTreats Framework 14

3.1.1 Knowledge Graph Generation 14

3.1.2 Error Tracking 15

3.1.3 Prompt Generation 15

3.1.4 Repair Attempt 15

3.2 Application to Verus 15

3.2.1 Selecting a Large Language Model 15

3.2.1.1 Dataset Construction 15

4 Findings **16**

4.1 Name of Subsection 16

4.2 Name of Subsection 17
4.2.1 Name of Third-Level Subsection 17
4.2.1.1 Name of Fourth-Level Subsection 17

5 Conclusion **18**
5.1 Name of Subsection 18
5.2 Name of Subsection 18
5.2.1 Name of Third-Level Subsection 18
5.2.1.1 Name of Fourth-Level Subsection 18

References **19**

A Title of Appendix **20**
A.1 Name of Subsection 20
A.2 Name of Subsection 20
A.2.1 Name of Third-Level Subsection 20
A.2.1.1 Name of Fourth-Level Subsection 20

B Title of Appendix **21**
B.1 Name of Subsection 21
B.2 Name of Subsection 21
B.2.1 Name of Third-Level Subsection 21
B.2.1.1 Name of Fourth-Level Subsection 21

List of Figures

2.1 Insert figure caption here 12

A.1 Insert figure caption here 20

List of Tables

4.1 Insert table caption here 16

4.2 Insert table caption here 16

B.1 Insert table caption here 21

Chapter 1

Introduction

Abstract

This is where you will insert your chapter abstract, which funcitons as a concise, self-contained summary of your chapter.

1.1 Overview

[What is Formal Methods and its significance?] The desire for high availability and secure software has only grown as we continue to rely on computers for critically important functions. Whether that be for banking services, pacemaker devices, or AWS a service which hosts [stupid amount of money] in business every second. For any of these cases even minor malfunctions could have significant consequences. Thus one might be interested in guaranteeing correct behavior for these systems. This is the goal of formal methods where we can formally prove properties about a program via [math...]. However formal methods has seen little use in practical applications as the process is unfamiliar to the typical programmer and time-intensive. [Reference a large formally verified program like C compiler project?]. [Introduce automated proof tools].

[Discuss past work in proof automation, then the rise of LLM based tools] This had led to the development of proof synthesis tools that attempt to automatically generate proofs

[Introduce our work in this field]. [Setup the problem we seek to address] When utilizing LLM's bluntly to solve an entire proof they either generate a correct solution or a completely wrong solution, in that no part of the proof is partially correct. In addition the LLM lacks knowledge of any Lemma's or definitions that have been defined in the local environment

[This method works best for theorem provers with underlying SAT solvers that attempt to solve the proof state]

Type out the content of your subsection here. What follows is an example APA 7th citation (Metz, 1978). Note also Metz, Einstein, et al. (1978) but Metz, Hamlin, et al. (1978). You can adjust citational information by editing the allcites.bib file, included in this package.

1.2 Background

The behavior of a program in an ideal world conforms exactly to the specification defined by the programmer. However whether due to mistakes, side effects, or the [scale of a project growing too large] this is rarely the case. Thus we might look for ways to make guarantees about a program; this is where formal methods arises. We are able to build rigorous statements about a system from the specifications that allow us to make guarantees [Specific kind?] about its behavior. Rigorous in this context means that there are formal rules for constructing new statements, this means any conclusions we make from the specification hold true [Stronger word for true? Sound].

There are two typical approach's taken to formal methods problems, either model checking or theorem proving. Model Checking is an automated procedure that attempts to search the entire state space of a system to verify a specification. The method is limited to finite systems and even then is prone to state-space explosion where the computational requirements grow exponentially with the size of the system making it infeasible on larger systems. Theorem proving on the other hand is more expressive as verification relies on constructing a proof that a property holds for the given system. This means we can use our techniques for reasoning with unbounded objects to prove properties about infinite state systems. However, this is reliant on the programmer being able to construct said proof, and while there do exist tools such as proof assistants like [list proof assistants; Rocq], the construction remains labor-intensive.

Verus is an SMT-based theorem prover specifically for the Rust programming language. It is built on top of Rust so that it is able to utilize the programming languages linear type system in order to make the SMT-based verification easier. The goal of Verus is to extend the amount of safe

code it is possible to write in Rust. The linear type system that Rust is built to support is incredible at keeping code memory safe without any additional runtime overhead. However this does limit the expressivity of the language as certain structures will violate the linearity demanded by the type system. Verus then allows programmers to verify code that Rust would find unsafe, or other functional properties that could prevent bugs.

It is important to note that the cost of formal verification is typically too high to make it feasible for use in typical programs [Find a paper for this]. Thus there has been an increased interest in the automation of theorem proving methods in order to make formal verification more viable. In particular with the rise of large language models (LLM's) many tools seek to utilize their ability to generate code to solve proofs. Indeed the use of LLM's for proof generation manages to avoid some of the problems that arise when LLM's are used for code generation; namely the amount of technical debt, that is bugs or a lack of scalability, that LLM's introduce into a program. Since the content of a proof is insignificant as long as it is solved, we can effectively ignore its "quality" meaning we can dismiss any concerns about technical debt.

[Alt Paragraph] It is important to note that the cost of formal verification is typically too high to make it feasible for use in typical programs [Find a paper for this]. Thus there has been an increased interest in the automation of theorem proving methods in order to make formal verification more viable. Now the rise of Large Language Models has provided a unique tool that appears to be particularly suited for proof generation. Large Language Models, or LLM's, are complex AI models trained on large sets of text data combined with the ability to consider large amounts of input. This has resulted in LLM's being capable of generating human sounding text or, more relevant for us, functional code. However the use of LLM's for code generation incurs a greater amount of technical debt than the typical developer. This is where we find it particularly useful for proof generation as a theorem is agnostic to the content of a proof, only caring if the proof holds or not, this means that any output from an LLM used in the proof will have no effect on any code outside the proof.

[Need information about LLM prompts; large prompts effecting results]

Chapter 2

Literature Review

Abstract

This is where you will insert your chapter abstract, which funcitons as a concise, self-contained summary of your chapter.

2.1 Name of Subsection

Type out the content of your subsection here (Einstein, 1905). Delete or duplicate as necessary. Note that you can also refer to figures in the text like this: see figure 2.1.

Figure 2.1: Insert figure caption here



2.1.1 Name of Third-Level Subsection

Here is an example of a third-level subsection. Delete or duplicate as necessary.

2.1.1.1 Name of Fourth-Level Subsection

Here is another example of a fourth-level subsection. Delete or duplicate as necessary.

Chapter 3

Methodology

Abstract

This is where you will insert your chapter abstract, which funcitons as a concise, self-contained summary of your chapter.

3.1 DogTreats Framework

Dogtreats is a method for effectively utilizing LLM's to generate the content of a proof, in order to automate part of the theorem proving process. However using LLM's alone to synthesize a full proof means that we could fail to reference objects in the local context that are either necessary or helpful for solving the proof. Furthermore in the case of failure diagnosing why the proof failed or finding a partial solution from the provided proof can be difficult. Thus our method categorizes and ranks existing local information by relevance to provide to the LLM, and then attempts to iteratively add and test single statements to the proof effectively constructing a partial solution at each step. The full process is given as follows: [How to transition into the full description?]

3.1.1 Knowledge Graph Generation

A knowledge graph for our purposes is a graph structure that relates an object O_1 to an object O_2

[Would a mathematical model of a knowledge graph be of benefit here]

3.1.2 Error Tracking

3.1.3 Prompt Generation

3.1.4 Repair Attempt

3.2 Application to Verus

Here is another example of a chapter subsection. Delete or duplicate as necessary.

3.2.1 Selecting a Large Language Model

Here is another example of a third-level subsection. Delete or duplicate as necessary.

3.2.1.1 Dataset Construction

Here is another example of a fourth-level subsection. Delete or duplicate as necessary.

Chapter 4

Findings

Abstract

This is where you will insert your chapter abstract, which funcitons as a concise, self-contained summary of your chapter.

4.1 Name of Subsection

Type out the content of your subsection here. Here's another citation (Einstein, 1905; Thomas, 1998). What follows is an example table:

Table 4.1: Insert table caption here

Set	Test 1	Test 2	Test 3	Test 4
1	50	34	837	970
2	47	877	34	230
3	5	31	25	415
4	35	27	144	2356
5	45	300	556	389

And just to be safe, here is another example of a table:

Table 4.2: Insert table caption here

Experiment	Batch A	Batch B	Batch C	Batch D	Batch E
5	7	5	3	7	2
8	9	8	2	1	9
3	2	1	9	6	6
2	2	8	1	4	9

You can also refer to tables in the text like this: see tables 4.1 and 4.2.

4.2 Name of Subsection

Here is another example of a chapter subsection. Delete or duplicate as necessary.

4.2.1 Name of Third-Level Subsection

Here is another example of a third-level subsection. Delete or duplicate as necessary.

4.2.1.1 Name of Fourth-Level Subsection

Here is another example of a fourth-level subsection. Delete or duplicate as necessary.

Chapter 5

Conclusion

Abstract

This is where you will insert your chapter abstract, which funcitons as a concise, self-contained summary of your chapter.

5.1 Name of Subsection

Type out the content of your subsection here. Delete or duplicate as necessary.

5.2 Name of Subsection

Here is another example of a chapter subsection. Delete or duplicate as necessary.

5.2.1 Name of Third-Level Subsection

Here is another example of a third-level subsection. Delete or duplicate as necessary.

5.2.1.1 Name of Fourth-Level Subsection

Here is another example of a fourth-level subsection. Delete or duplicate as necessary.

References

Einstein, A. (1905). Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, *322*(10), 891–921. https://doi.org/10.1002/andp.19053221004

Metz, C. E. (1978). Basic principles of ROC analysis. *Seminars in Nuclear Medicine*, *8*, 283–298.

Metz, C. E., Einstein, A., Thomas, P. A., Benedict, A. B., & Rice, C. (1978). Principles of potentially redundant citations. *Bibliographic Studies*, *34*, 56–78.

Metz, C. E., Hamlin, H., Thomas, P. A., Boss, H., Benedict, A. B., & Redfield, C. R. (1978). A meta-analysis of redundant citations. *Bibliographic Studies*, *34*, 79–84.

Thomas, L. (1998). Statistical power analysis software. http://www.forestry.ubc.ca/conservation/power/

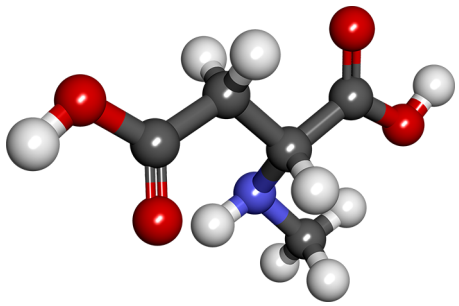
Appendix A

Title of Appendix

A.1 Name of Subsection

This is an example of an appendix. You can put citations here (Metz, 1978), as well as figures:

Figure A.1: Insert figure caption here



A.2 Name of Subsection

Here is another example of an appendix subsection. Delete or duplicate as necessary.

A.2.1 Name of Third-Level Subsection

Here is another example of a third-level appendix subsection. Delete or duplicate as necessary.

A.2.1.1 Name of Fourth-Level Subsection

Here is another example of a fourth-level appendix subsection. Delete or duplicate as necessary.

Appendix B

Title of Appendix

B.1 Name of Subsection

This is an example of an appendix. Just like chapters, you can also include tables in appendices:

Table B.1: Insert table caption here

Experiment	Batch A	Batch B	Batch C	Batch D	Batch E
6	9	5	3	7	2
8	3	8	2	1	9
9	7	1	9	6	6
9	4	8	1	4	9

B.2 Name of Subsection

Here is another example of an appendix subsection. Delete or duplicate as necessary.

B.2.1 Name of Third-Level Subsection

Here is another example of a third-level appendix subsection. Delete or duplicate as necessary.

B.2.1.1 Name of Fourth-Level Subsection

Here is another example of a fourth-level appendix subsection. Delete or duplicate as necessary.